FULL-STACK ENGINEERING

全栈技术栈系统学习路线

以核心概念覆盖率与独立交付能力为标准，而不是以学习时长为标准

技术范围

React、TypeScript、Vite、React Router Data Mode、TanStack Query、Zustand、shadcn/ui（Base UI）、React Hook Form、Zod、Tailwind CSS；Node.js、Fastify、TypeBox / JSON Schema、OpenAPI、Drizzle ORM、Better Auth、Swagger UI / Scalar、结构化日志；Supabase；pnpm、Turborepo、Docker、GitHub Actions、Vitest、Playwright、Sentry、OpenTelemetry。

文档版本：2026-10-22 | 路线类型：项目驱动、可验收、可复盘

本路线的核心原则

每个技术都要完成四件事：讲清核心机制、从空白项目独立实现、为主要失败路径编写测试、遇到异常时能定位原因。达到“90%核心掌握”不等于读完 90%的文档，而是能对本路线的核心清单逐项提供可验证证据。

你最终要获得的能力

- 能独立设计并交付一个有登录、权限、业务数据、表单、筛选分页、测试、CI/CD 与可观测性的全栈应用。
- 能说清数据从浏览器到 API、数据库，再回到 UI 的完整生命周期与信任边界。
- 能分辨路由状态、服务端状态、全局 UI 状态、表单状态与持久化数据，不把所有状态塞进同一个工具。
- 能为团队建立可复现的开发环境、数据库迁移流程、自动化质量门禁与故障排查方式。

如何使用这份路线图

顺序按前置依赖与系统集成难度安排，不按技术热度，也不按固定日程安排。每完成一个阶段，先通过验收，再进入下一阶段；如果只是跟着教程敲出结果但不能脱离教程重做，不算通过。

90%核心掌握的操作性定义

维度	最低证据	不算掌握的情况
概念覆盖	一个技术的核心清单至少 90% 项能用自己的话解释，并能说明适用边界。	只记住 API 名称；说不清机制与取舍。
独立实现	从空文件或干净分支实现核心场景，不复制整段教程。	必须逐步照着视频/文章才能继续。
失败路径	主动覆盖错误输入、未授权、超时、并发/重复提交、空结果等场景。	只验证 happy path。
调试与取舍	能读日志、复现问题、缩小原因范围，并说明另一种实现为什么不选。	出现问题只尝试升级依赖或删缓存。
工程证据	有测试、README/决策记录、可复现的运行方式。	代码能在自己电脑跑，但别人无法重现。

红线：安全与数据正确性不能按 90% 放行

认证、授权、RLS / 数据隔离、参数化查询、密钥管理、迁移可回滚/可复现、错误信息不泄漏等安全关键项必须全部通过；其余核心清单可以用“至少 90% + 项目验收”作为阶段门槛。

推荐的项目结构目标

```
repo/
  apps/
    web/                # React + Vite 单页应用
    api/                # Fastify API
  packages/
    db/                 # Drizzle schema / queries / migrations ownership
    api-client/         # 由 OpenAPI 生成的客户端类型或请求封装
    ui/                 # 可复用 UI（按真实需要建立）
    config/             # tsconfig / lint / shared tooling
  supabase/             # 本地环境配置、seed；迁移策略需唯一
  e2e/                  # Playwright 场景
  turbo.json
  pnpm-workspace.yaml
  package.json
```

不要为了“用 monorepo”而过早拆出十几个 package。最小合理结构是 web、api、db 与 api-client；只有当共享代码有明确边界和使用方时，才抽出 ui/config 等包。

总路线鸟瞰

阶段	主题	阶段产物
0	Web / JS / TS / Git / SQL 基础	能调试浏览器、写类型安全的代码、理解关系型数据。
1	React + Vite + Tailwind	有组件结构、样式规范与生产构建的单页应用。
2	React Router Data Mode	嵌套路由、loader/action、错误边界与待处理状态。
3	TanStack Query + Zustand	服务端缓存与客户端 UI 状态分工清晰。
4	shadcn/Base UI + RHF + Zod	可访问、校验可靠、状态完整的复杂表单。
5	Node.js + Fastify + TypeBox + OpenAPI	有 schema、错误规范、日志和可交互 API 文档的后端。
6	Supabase/Postgres + Drizzle	有规范 schema、迁移、索引、权限与数据访问层。
7	Better Auth + 安全集成	登录、会话、授权、跨域与权限测试闭环。
8	Vitest + Playwright	单元/集成/E2E 测试覆盖关键业务路径。
9	pnpm + Turborepo + Docker + GitHub Actions	可复现 monorepo、容器化与 CI 质量门禁。
10	Sentry + OpenTelemetry	从前端错误追踪到后端日志与 trace 的排障闭环。
11	毕业项目与复盘	完整全栈应用、架构决策记录与掌握度证据。

第一部分｜基础能力：先建立共同语言

阶段 0｜Web、JavaScript、TypeScript、Git 与 SQL 基础

目标：能够独立读写常见现代 Web 代码，并理解前后端共同依赖的基本机制。

核心概念清单（逐项验收）

- ☐ JavaScript：词法作用域、闭包、对象/数组引用、解构、可选链、模块、异常、Promise、async/await、事件循环的基本心智模型。
- ☐ 异步：并发与顺序、Promise.all / allSettled、超时与取消（AbortController）、错误传播、网络请求失败分类。
- ☐ TypeScript：基础类型、联合/交叉、字面量类型、narrowing、泛型、utility types、函数重载的基本使用、unknown 与 never。
- ☐ 类型设计：区分编译期类型与运行期校验；理解类型断言不会验证外部数据；避免滥用 any。
- ☐ Web：HTTP 方法/状态码、请求头、Cookie、CORS、缓存、浏览器存储、URL/query string、HTML 表单、基本可访问性。
- ☐ CSS：盒模型、display、position、flex/grid、响应式布局、层叠/优先级、焦点样式与语义化 HTML。
- ☐ Git：分支、commit、merge/rebase 基本语义、冲突处理、回滚、.gitignore；能检查 diff。
- ☐ SQL：表、主外键、唯一/非空/check 约束、JOIN、GROUP BY、索引、事务、NULL 语义、分页与排序的确定性。

必须完成的动手练习

1. 写一个带类型的 HTTP 请求函数：解析响应、处理非 2xx、支持 AbortSignal，并测试超时/失败。
2. 写一组纯函数，将未知 JSON 转为经过校验的业务值；比较“类型断言”和“运行时验证”的区别。
3. 在 Postgres 兼容环境中创建 users/projects/tasks 三张表，写 JOIN、聚合查询、唯一约束与事务练习。
4. 制作一页键盘可操作的表单，使用浏览器开发者工具检查网络请求、响应头、Cookie 和 DOM。

通过标准

- 不用查教程，能解释 Promise 链/async-await 的错误处理，以及为什么网络数据默认是 untrusted input。
- 能阅读并修复至少一种复杂联合类型或 narrowing 问题，不靠 as any 绕过编译器。
- 能用 SQL 解释主键、外键、唯一约束、索引和事务分别解决什么问题。
- 能用 Git 解决一次真实冲突，并说明如何检查提交中是否混入密钥。

边界 / 易错点

TypeScript 类型不替代运行时校验；前端传入的数据、环境变量和第三方 API 响应均应视为外部输入。

第二部分｜前端：让状态流向有清晰边界

阶段 1｜React + TypeScript + Vite

目标：构建组件化 SPA，掌握 React 渲染模型与开发/构建工具链。

核心概念清单（逐项验收）

- ☐ 组件、JSX、props、children、组合优于复杂继承；列表 key 应稳定且表达业务身份。
- ☐ state 与渲染快照、事件处理、受控输入、状态提升、派生状态；避免通过重复 state 造成不同步。
- ☐ Hooks 规则；useState、useReducer、useRef、useEffect、useContext 的正确使用场景。
- ☐ Effect 用于同步外部系统，不是每次渲染后的通用计算钩子；清理订阅、计时器和请求。
- ☐ useMemo/useCallback/memo 的收益与成本；先测量再优化。
- ☐ 错误边界、Suspense 的用途与限制；加载、空、错误、成功四类 UI 状态。
- ☐ TypeScript 组件 props、事件类型、泛型组件、children、可辨别联合状态建模。
- ☐ Vite：dev server、HMR、生产 build、环境变量前缀与暴露范围、静态资源、alias、插件与配置。
- ☐ 打包与浏览器缓存、动态 import / route-level code splitting、source map 的基础概念。

必须完成的动手练习

1. 做一个可搜索的任务列表，拆出列表、行、筛选条、空状态、错误状态等组件。
2. 将重复的请求/订阅写法改成有 cleanup 的 Effect，并补充卸载或快速切换条件的测试。
3. 用 Vite 构建后预览 production build，验证环境变量、资源路径和路由刷新。
4. 选一个组件，分别用 props、状态提升和组合方式重构一次，解释三者取舍。

通过标准

- 组件状态变化时能预测哪些组件会重新渲染，并用 React DevTools 验证。
- 能从空白 Vite 项目搭建 React + TS，不需要模板教程逐步指引。
- 能区分构建期环境变量和运行时秘密；没有把服务端密钥放进 Vite 客户端变量。
- 生产构建可通过，不依赖开发服务器功能。

边界 / 易错点

不要把 useEffect 当作“同步所有变量”的万能工具；能在 render 中计算的值通常不需要再存一份 state。

阶段 2｜Tailwind CSS

目标：形成可维护的 utility-first 样式方式，并能在响应式、主题与组件状态下保持一致。

核心概念清单（逐项验收）

- ☐ utility-first 思想：将布局、尺寸、排版、颜色、边框、阴影、间距映射为组合类名。
- ☐ 响应式断点、hover/focus/active/disabled、group/peer 等状态变体。
- ☐ 深色模式与主题令牌；设计 token 与语义颜色，而不是散落硬编码色值。
- ☐ flex/grid、溢出、定位、z-index、容器宽度、移动端布局。

- ☐ className 条件组合、可复用组件 API 与样式职责划分。
- ☐ 当前 Tailwind 版本的 CSS-first 配置与旧版配置差异；项目必须跟随所锁定主版本文档。
- ☐ 避免动态拼接无法被构建工具识别的 class；必要时使用明确映射或 safelist。
- ☐ 基础 CSS 仍需理解：utility 不会替代语义、可访问性和层叠模型。

必须完成的动手练习

1. 为任务面板做桌面、平板、手机三种布局，包含侧栏折叠、任务卡片与筛选栏。
2. 建立颜色/间距/圆角的最小设计 token，并为暗色模式验证对比度。
3. 对按钮、输入框、弹窗做 hover、focus-visible、disabled 和错误状态。

通过标准

- 不查类名表也能实现常见 Flex/Grid 布局，并能用 DevTools 找到溢出来源。
- 可解释组件抽取和直接堆叠 utility 的边界；重复模式已抽出而非过度抽象。
- 页面键盘焦点清楚，文本与控件对比度基本满足可访问性要求。

边界 / 易错点

Tailwind 主版本之间配置方式会变化；不要把旧教程的配置文件、指令或插件写法直接移植到新项目。

阶段 3｜React Router（Data Mode）

目标：能将页面层级、URL、加载/提交数据与导航状态组织成清晰的路由系统。

核心概念清单（逐项验收）

- ☐ Data Mode 与 Declarative / Framework Mode 的差异；理解选定 Data Mode 的原因。
- ☐ createBrowserRouter、RouterProvider、嵌套路由、layout route、index route、动态参数。
- ☐ loader：路由进入前读取数据、params/search params、重定向与请求取消。
- ☐ action：表单提交、服务端错误映射、redirect、提交状态。
- ☐ useNavigation、useFetcher、pending UI、并行加载与避免重复请求。
- ☐ errorElement / 路由错误边界、not found、权限拒绝和意外异常的区分。
- ☐ useNavigate、Link、useSearchParams；将可分享的筛选、排序、分页放入 URL。
- ☐ route lazy / 动态导入、路由级代码拆分、loader/action 的单元测试方式。

必须完成的动手练习

1. 建立 /login、/app、/workspaces/:id、/projects/:id、/tasks/:taskId 的嵌套路由。
2. 让筛选和排序同步到 query string，刷新或复制 URL 后结果保持一致。
3. 实现 loader 失败、无权限、资源不存在和 action 校验失败的不同 UI。

通过标准

- 能解释哪些数据适合 loader、哪些适合 TanStack Query，并让同一份数据避免无意义双重请求。
- URL 是页面状态的可恢复来源：筛选、分页、排序刷新后可重建。
- 导航待处理、表单提交中和路由错误都能给出可理解的反馈。

边界 / 易错点

可选架构：由 Router loader 完全管理页面数据，或在 loader 中调用 queryClient.ensureQueryData 统一进 TanStack Query 缓存。避免 loader 与 useQuery 各自独立请求同一资源。

阶段 4｜TanStack Query + Zustand：状态分层

目标：分别掌握服务端状态缓存与客户端 UI 状态，能解释为什么不应把它们混成一种状态。

核心概念清单（逐项验收）

- ☐ TanStack Query：QueryClient、query key 结构、queryFn、useQuery、loading/error/success 状态。
- ☐ 缓存新鲜度 staleTime、垃圾回收 gcTime、失效 invalidateQueries、主动 refetch 与默认自动刷新。
- ☐ useMutation、mutation 生命周期、成功后失效相关 query、取消请求、重试和错误处理。
- ☐ 乐观更新：先改缓存、失败回滚、并发 mutation 与最终一致性风险。
- ☐ 分页、无限滚动、依赖查询、预取、后台刷新；理解缓存 key 中必须包含所有影响结果的变量。
- ☐ 结构共享、数据引用稳定性、Query Devtools 与默认重试/刷新行为。
- ☐ Zustand：store/action/selector、细粒度订阅、derived state、多个 store 的拆分策略。
- ☐ Zustand persist：partialize、版本升级与 migrate、持久化存储不应保存秘密。
- ☐ 测试与调试：每个测试隔离 QueryClient；不要把服务端数据复制一份进 Zustand。

必须完成的动手练习

1. 为项目/任务列表建立 query key 工厂，覆盖 workspace、筛选、排序、分页参数。
2. 实现“新增任务、更新状态、删除任务”三种 mutation，正确刷新关联列表和详情。
3. 用 Zustand 存储侧栏开关、当前打开的 drawer、临时选择项等纯 UI 状态。
4. 故意构造错误缓存 key，并观察数据串用；再通过 Devtools 找出问题。

通过标准

- 能说明服务端数据为何交给 Query；本地 UI 偏好为何适合 Zustand；URL 状态为何交给 Router。
- 更新后列表、详情、计数器等相关视图一致；失败时用户收到反馈且缓存不被污染。
- 没有在 Zustand 中再存一份 API 数据，也没有把可从 props/state 推导出的值重复存储。

边界 / 易错点

TanStack Query 的缓存默认值并非“永不刷新”；要理解 staleTime、gcTime、invalidate 与窗口聚焦/网络恢复时的行为。

阶段 5｜shadcn/ui（Base UI）+ React Hook Form + Zod

目标：能构建可访问的设计系统组件和复杂表单，并建立客户端校验、服务端校验与错误显示闭环。

核心概念清单（逐项验收）

- ☐ shadcn/ui 是可被拷入并修改的组件源码/分发方式；维护组件源码、tokens、variants 与自定义边界。

- ☐ Base UI primitives：受控/非受控模式、组合式 API、弹窗/菜单/Popover/Select、焦点管理与键盘行为。
- ☐ 可访问性：label 与控件关联、ARIA 状态、错误描述、focus trap、escape 关闭、返回焦点。
- ☐ React Hook Form：register、Controller、useForm、formState、defaultValues、reset、watch、setValue。
- ☐ 表单生命周期：初始化、dirty/touched、提交中、成功后重置、服务端错误与重复提交防护。
- ☐ 字段数组、嵌套对象、条件字段、跨字段校验、异步校验与字段级错误。
- ☐ Zod：object、string/number/enum、optional/nullable/default、refine/superRefine、transform、safeParse。
- ☐ z.input 与 z.output；转换前输入与解析后输出类型可能不同。
- ☐ 使用 zodResolver 接入 RHF；客户端校验是用户体验，不是安全边界。

必须完成的动手练习

1. 做“创建/编辑任务”表单，包含标题、描述、优先级、负责人、截止日期和条件字段。
2. 覆盖空值、字符长度、非法日期、服务端权限错误、网络失败与重复点击提交。
3. 从 shadcn 源码修改一个 Base UI 组件的样式/行为，并验证键盘操作未被破坏。
4. 使用屏幕阅读器友好的错误文本与 aria-invalid，键盘操作完成一遍完整表单。

通过标准

- 无论何时提交，服务端都会重新校验输入并进行授权。
- 表单不会在失败时丢失所有输入；提交中禁用重复动作，成功后按业务意图 reset / navigate。
- 所有关键控件有 label，错误和帮助文本能被辅助技术关联；Select/Dialog 等键盘交互正常。

边界 / 易错点

Base UI 与 Radix UI 的底层 primitive 不同。不要直接照搬另一套 primitive 的 props 或事件约定；以项目选定版本的 shadcn/Base UI 组件源码为准。

第三部分｜后端与数据：让契约、权限和持久化可验证

阶段 6｜Node.js 工程基础

目标：理解 Fastify 运行于何种执行模型，以及服务进程如何配置、退出和处理异步错误。

核心概念清单（逐项验收）

- ☐ ESM/CJS、package.json、Node.js 与浏览器运行环境的差异、process.env 与配置注入。
- ☐ 事件循环基本概念、异步 I/O、Promise rejection 与异常传播；不要在请求路径中做长时间阻塞工作。
- ☐ 模块边界、依赖注入、配置验证、日志与错误分层。
- ☐ HTTP Server 生命周期、信号处理、优雅关闭、健康检查、连接池释放。
- ☐ 流与 Buffer 的基本用途；文件上传/大文件处理不要无界读入内存。
- ☐ 安全基线：依赖审计、路径处理、大小限制、超时、限流、环境变量与密钥管理。

必须完成的动手练习

1. 创建一个最小 Node.js API 进程，验证 ESM、环境变量校验、统一错误出口。
2. 给模拟的慢任务加超时和 AbortSignal，比较同步阻塞与异步 I/O。
3. 实现 SIGTERM 下的优雅关闭，确保数据库连接池不会被强制遗留。

通过标准

- 知道 Node.js 进程与浏览器执行环境的关键差异。
- 缺失必需环境变量时进程应启动失败并给出明确错误，而不是直到线上请求才暴露。
- 退出、重启、错误处理与健康检查行为可重复验证。

阶段 7｜Fastify + TypeBox / JSON Schema + OpenAPI + Swagger UI / Scalar

目标：构建有清晰插件边界、运行时输入输出 schema、稳定错误模型、结构化日志与 API 文档的 HTTP API。

核心概念清单（逐项验收）

- ☐ Fastify 实例、register、插件封装/作用域、decorator、route、request/reply 与 hook 生命周期。
- ☐ preValidation / preHandler / onRequest / onResponse 等 hook 的职责与执行顺序。
- ☐ JSON Schema 校验 params/querystring/headers/body；用 response schema 控制输出形状并避免意外泄露字段。
- ☐ TypeBox 的 Type、Static、Optional、Union、Literal、引用与可复用 schema；类型推断与运行时 schema 同源。
- ☐ schema $id、复用与封装范围；区分结构校验和需要访问数据库的业务规则。
- ☐ 统一错误处理：业务错误、校验错误、未授权、禁止访问、资源不存在、冲突、限流和未知错误。
- ☐ HTTP 状态码、分页/排序契约、请求 ID、稳定错误 code 与可安全暴露的信息。
- ☐ @fastify/swagger 的注册顺序、schema 到 OpenAPI 的映射、API 版本与破坏性变更。
- ☐ Swagger UI 与 Scalar 都是 API reference 前端，选择其一即可；文档环境应限制或只暴露安全信息。
- ☐ Fastify 内置 Pino 日志、子 logger、结构化字段、日志级别、敏感字段 redact。
- ☐ 通过 app.inject 进行路由集成测试；测试插件装配和请求/响应行为。

必须完成的动手练习

1. 实现 /health/live、/health/ready、GET/POST/PATCH/DELETE /api/v1/tasks。
2. 所有请求参数与响应均定义 schema；生成 OpenAPI 文档，并用 Scalar 或 Swagger UI 调试。
3. 构建统一错误格式 { error: { code, message, requestId } }，保证异常路径不输出 stack/SQL/密钥。
4. 使用 Fastify inject 覆盖 2xx、400、401、403、404、409、500 等关键路径。

通过标准

- 非法输入在进入业务逻辑前被拒绝；业务/授权检查不混进 schema validator 中查询数据库。
- 响应 schema 可阻止意外返回内部字段；所有接口有明确状态码和错误契约。
- API 文档与实际路由同步，客户端无需读后端实现就能理解参数与返回值。
- 日志是 JSON 结构，包含 requestId、路由、耗时、状态码；不记录密码、cookie、token、完整个人敏感信息。

边界 / 易错点

Fastify 的 JSON Schema 主要用于运行时校验与序列化；TypeScript 类型本身不会在网络边界自动生效。初始校验阶段不要为每条请求执行任意数据库查询。

阶段 8 | Supabase 与 PostgreSQL

目标：把 Supabase 理解为托管 Postgres 及周边开发/部署能力，而不仅是一个控制台或 SDK。

核心概念清单（逐项验收）

- ☐ Postgres：关系模型、主外键、约束、事务与隔离级别的基础、锁与并发更新的基本风险。
- ☐ 表设计与范式：多对多 join table、唯一约束、外键删除策略、软删除的成本。
- ☐ 索引：B-tree 常见用途、复合索引顺序、查询计划与 EXPLAIN；索引不是越多越好。
- ☐ SQL：JOIN、聚合、子查询/CTE、窗口函数的基本理解、NULL、排序和稳定分页。
- ☐ Supabase CLI、本地开发栈、config.toml、migration、seed、reset 与远程部署流程。
- ☐ 项目环境隔离：local / preview / production，环境变量和数据库凭据按环境分开。
- ☐ RLS policy：启用与未启用的区别、USING / WITH CHECK、按用户/组织限制读写。
- ☐ 数据库 API/客户端直连与服务端直连的权限模型；service role / secret key 只能放服务器。
- ☐ 连接池、direct connection、transaction pooler 的差异；根据运行环境与连接方式选择。
- ☐ Storage bucket、文件权限、生命周期、签名 URL 的适用场景；不要把公开桶当成私有数据存储。
- ☐ 备份、恢复、迁移前后验证、数据种子与个人/线上数据保护。

必须完成的动手练习

1. 用 migration 创建 workspace、workspace_member、project、task、comment 表及约束。
2. 为“用户只能访问自己所属 workspace”的场景设计并测试 RLS policy。
3. 用 EXPLAIN 检查任务列表查询；在数据量增长后验证索引是否匹配查询条件。
4. 使用本地 Supabase 环境验证从干净数据库到完整 schema + seed 的可重建流程。

通过标准

- 能从零创建本地数据库，并让新同事通过仓库内容重建 schema 和测试数据。
- 所有租户边界都有数据库约束/授权策略或可审查的服务端授权，且用跨用户测试验证失败路径。
- 不在前端暴露 service role、数据库密码或 Better Auth 的 secret。
- 能解释所选 DB connection/pooler 的原因，并知道迁移脚本应在哪里运行。

边界 / 易错点

Supabase RLS 只有在对应表/查询路径正确启用并配置 policy 时才提供隔离。若业务数据全部由 Fastify + Drizzle 访问，服务端授权仍必须存在；不要因为有 RLS 就省略 API 授权，也不要以 service-role key 绕过后再假定 RLS 会保护请求。

阶段 9 | Drizzle ORM

目标：能把 SQL 数据模型映射到类型化的数据访问层，并理解 ORM 不会替代数据库设计。

核心概念清单（逐项验收）

- ☐ Postgres column builders、nullable/default、primary/foreign key、unique/index/check constraint。
- ☐ Drizzle schema 与 TypeScript inferred select/insert types；insert shape 与 select shape 的区别。
- ☐ 关系定义与数据库外键的区别；一对多、多对多、relation query 与 SQL join。
- ☐ Select/insert/update/delete、条件组合、排序/分页、返回字段投影与类型推断。
- ☐ 事务 transaction、批量写入、事务边界、并发一致性、唯一冲突处理。
- ☐ drizzle-kit：generate/migrate、schema diff、迁移历史、生产环境迁移与回滚策略。
- ☐ raw SQL 与参数绑定、避免 SQL injection、检查 N+1 与无界查询。
- ☐ 连接池配置、Supabase Postgres 连接方式、prepared statements 与部署环境限制。
- ☐ Repository/Service 的边界；不要为每个 SQL 包一层没有价值的抽象。

必须完成的动手练习

1. 将 workspace/project/task schema 用 Drizzle 表达，并生成 select/insert 类型。
2. 写一个事务，在创建项目时同时创建默认状态/初始设置；模拟第二步失败，验证整体回滚。
3. 实现带筛选的分页列表、详情 join、更新状态与删除，并为查询添加合适索引。
4. 从空数据库执行生产迁移脚本，然后跑 smoke test。

通过标准

- 业务规则放在 Service 层，数据库操作放在可测试的 query/repository 边界，但不过度抽象。
- 能解释数据库约束和 TypeScript 类型各自保护什么。
- 迁移流程可重放，并已选定单一 schema/migration source of truth。
- 所有动态筛选都使用参数化条件；不通过字符串拼接把用户输入写入 SQL。

边界 / 易错点

Drizzle 与 Supabase CLI 可以共存，但迁移职责必须清楚。选一个作为 schema 变更的主要来源：例如 Drizzle migrations 管理业务表，Supabase CLI 管本地环境/seed；或以 Supabase SQL migrations 为唯一来源并明确同步生成类型。不要两边独立修改同一张表。

阶段 10 | Better Auth 与授权模型

目标：理解认证、会话和授权的不同职责；把 Better Auth 安全地接进 Fastify 与 Postgres。

核心概念清单（逐项验收）

- ☐ 认证 Authentication 与授权 Authorization：已登录不代表有权访问某个 workspace/task。
- ☐ Better Auth 实例、数据库适配、用户/账户/会话数据模型、Fastify handler 集成。
- ☐ Cookie-based session、到期/续期/撤销、trusted origins、安全 cookie 属性与 session 生命周期。
- ☐ CORS 与 credentials、跨子域/跨站部署约束、CSRF 风险与框架推荐配置。
- ☐ 邮箱密码、邮箱验证、重置密码、OAuth 等能力按需求启用；邮件交付和回调 URL 需要真实配置。

- ☐ 密码哈希、登录速率限制、枚举风险、暴力破解防护、会话撤销和安全事件。
- ☐ 在每条受保护 API 请求中验证会话，并在业务层检查资源/租户成员关系。
- ☐ RBAC / ownership：角色、资源归属、组织成员关系、最小权限和默认拒绝。
- ☐ Better Auth 的表结构/migrations 与 Drizzle 自有 schema 的边界；升级前后如何检查迁移。
- ☐ 测试未登录、过期 session、非成员、只读角色、跨 workspace 访问和管理员行为。

必须完成的动手练习

1. 实现注册/登录/登出/当前用户接口；浏览器刷新后会话状态能按预期恢复。
2. 为 task CRUD 添加 workspace membership 授权，而不仅是验证是否登录。
3. 补齐 origin/CORS/cookie 配置，验证开发与生产域名下的登录行为。
4. 建立跨用户攻击测试：尝试读取或修改另一个 workspace 的 task，必须被拒绝。

通过标准

- 登录凭据、Cookie 和 session token 不进入客户端日志/服务端普通日志。
- 受保护接口默认拒绝未授权访问；每个资源查询都受到 user/workspace scope 限制。
- 登录/登出/会话过期路径有自动化测试，cookie 与 CORS 配置由环境驱动。
- 能解释 Better Auth 用户身份与业务表 userId 的关联方式、删除/禁用用户的处理策略。

边界 / 易错点

对于这个技术组合，推荐让 Better Auth 负责认证与会话、Fastify 负责 API 授权、Supabase 托管 Postgres 存储数据。不要同时无设计地启用 Better Auth 和 Supabase Auth 作为两套身份源；若确需两者并存，必须明确 token/session 转换、用户 ID 映射与权限同步。

阶段 11｜API 合约与端到端数据流整合

目标：将前端和后端通过稳定可验证的契约连接起来，减少手写类型漂移。

核心概念清单（逐项验收）

- ☐ 以 Fastify route schema / OpenAPI 作为 API 结构的权威描述；文档包含请求、响应、状态码、安全方案和分页。
- ☐ 从 OpenAPI 生成客户端类型或请求封装，保持 apps/web 不直接依赖服务端实现文件。
- ☐ 前端 Zod 与后端 TypeBox 的职责：表单 UX 校验和服务端信任边界校验各自独立。
- ☐ 统一错误 envelope、requestId、验证错误字段映射、取消请求与网络重试策略。
- ☐ 认证 cookie、credentials、CSRF/CORS、浏览器预检请求与代理配置。
- ☐ 幂等与重复提交：支付类/创建类动作应根据业务定义是否需要 idempotency key；常规更新也需考虑并发。
- ☐ API 版本兼容、弃用策略、分页响应契约、日期/时区与枚举的序列化规范。

必须完成的动手练习

1. 生成或更新 OpenAPI 后运行客户端类型生成脚本，并让 CI 检查生成结果是否过期。
2. 实现一个完整“创建任务”链路：React Hook Form → Zod → Query mutation → Fastify schema → AuthZ → Drizzle → Postgres。

3. 将 400 字段错误、401、403、404、409 和 500 映射到合适的 UI。

通过标准

- 前端代码不复制维护第二套 API 响应类型；合同变更会在编译或 CI 中暴露。
- 任何客户端校验绕过都不能破坏服务器端的数据正确性与权限。
- 错误可被 UI 解释，同时服务器端日志保留排查所需上下文，不泄露内部细节。

第四部分｜质量保障：用测试证明系统行为

阶段 12｜Vitest

目标：为纯函数、React 组件、API 与数据库服务建立快速、可靠、低耦合的测试层。

核心概念清单（逐项验收）

- ☐ 测试层级：unit / component / integration；各自要证明的行为与成本。
- ☐ Vitest 配置、test environment、setupFiles、mock、vi.fn/spyOn、fake timers 与 cleanup。
- ☐ AAA（Arrange-Act-Assert）、边界值、参数化测试、异步断言和错误路径。
- ☐ React Testing Library：优先按 role/label/user-visible behavior 查询，而不是依赖实现细节。
- ☐ 测试隔离：每个测试自己的 QueryClient、状态 store、数据库事务/数据集。
- ☐ Fastify app.inject：无须监听外部端口也能覆盖请求/响应与插件行为。
- ☐ Mock 边界： mock 外部依赖但不要把被测业务逻辑全部 mock 掉；integration test 验证真实集成。
- ☐ Coverage 的 statements/branches/functions/lines 含义；覆盖率只是信号，不等于质量。
- ☐ 测试运行稳定性、随机顺序/并发、依赖清理与失败定位。

必须完成的动手练习

1. 为 Zod schema、权限判断、分页参数、日期转换、状态迁移等纯逻辑写边界测试。
2. 为表单组件测试必填错误、提交成功、服务器拒绝与 disabled 状态。
3. 为 Fastify API 测试验证错误、响应 schema、鉴权和统一错误处理。
4. 跑一次 coverage，找出分支漏测并解释为什么补的每条测试有价值。

通过标准

- 测试不依赖执行顺序、不因共享 Query cache/store 而互相污染。
- 重要错误分支和权限规则有明确断言，而非只测试默认成功路径。
- 能用测试快速定位回归，并避免过度 mock 造成“所有测试通过但系统无法运行”。

阶段 13｜Playwright 端到端测试

目标：从真实浏览器视角验证关键用户旅程，理解隔离、可靠等待与诊断产物。

核心概念清单（逐项验收）

- ☐ test/expect、page/context/browser、locator 与语义化定位（getByRole、getByLabel、getByTestId 的取舍）。

- ☐ 自动等待与 web-first assertions；为什么固定 sleep、任意 timeout 会制造脆弱测试。
- ☐ Fixture、beforeEach/afterEach、测试隔离、稳定的测试数据与并行执行。
- ☐ 配置 webServer、baseURL、浏览器项目、重试、trace、截图/视频与 CI 报告。
- ☐ 认证状态 storageState 的安全使用；不同角色的测试身份与数据种子。
- ☐ 使用 API/setup 准备测试数据，但真正的 E2E 仍要通过 UI 验证用户可观察行为。
- ☐ 网络失败、权限失败、加载状态、表单校验、分页与键盘交互。
- ☐ 失败诊断：trace viewer、控制台错误、请求失败、服务器日志和 requestId。

必须完成的动手练习

1. 自动跑通登录 → 创建 workspace/project/task → 修改状态 → 搜索/筛选 → 登出。
2. 增加跨 workspace 越权场景，验证用户看不到其他租户的数据。
3. 在 CI 中执行 E2E，失败时上传 trace 和必要截图；不要上传包含真实密钥/个人数据的状态文件。

通过标准

- 关键旅程无任意固定 sleep，主要定位使用可访问角色或明确测试 id。
- 测试数据相互隔离，可重复执行，不依赖生产服务或真实用户数据。
- 发生失败时能从 trace 还原页面动作、请求/响应与断言失败原因。

边界 / 易错点

Playwright 重试可以帮助诊断偶发问题，但不能把 flaky 测试当成正常现象。先查竞争条件、共享测试数据、网络依赖和不稳定等待，再决定是否调整 retry。

第五部分｜工程化、部署和可观测性

阶段 14｜pnpm + Turborepo Monorepo

目标：把 web、api、共享 schema/类型与测试组织到可复现、可缓存的仓库结构中。

核心概念清单（逐项验收）

- ☐ pnpm workspace：pnpm-workspace.yaml、workspace:*、lockfile、过滤器、依赖提升/隔离语义。
- ☐ Node/package 管理：package scripts、核心依赖版本锁定、包导出、ESM、tsconfig project references（按需）。
- ☐ Turborepo：task graph、dependsOn、outputs、cache、环境变量建模、任务边界与远程缓存。
- ☐ 每个 package 的职责：apps 存应用；packages 只放清晰可复用的模块；避免循环依赖。
- ☐ 共享 config 与共享业务逻辑的区别；不要将仅在服务器使用的数据库/密钥代码导入前端。
- ☐ OpenAPI 生成客户端任务的依赖顺序；build、typecheck、lint、test、e2e 任务图。
- ☐ CI 缓存与 Turborepo 缓存的区别；缓存命中必须以依赖、输入文件和相关环境为前提。
- ☐ workspace filters 在单包开发与全仓构建中的用法。

必须完成的动手练习

1. 把 web/api 拆成 workspace 包，统一 pnpm lockfile 与脚本。

2. 配置 turbo 任务：lint/typecheck/test/build，并声明正确的 dependsOn 与 outputs。
3. 将 OpenAPI client 生成加入依赖图，验证 schema 变化后 client 与 web 会重新检查。
4. 人为改动某个 package，观察缓存命中/失效，解释输入文件与环境变量的影响。

通过标准

- 新 clone 后用固定命令安装、启动、测试，无需手工在多个目录装依赖。
- 单包修改不必无条件重建全仓；但受影响的依赖包会被正确重新构建。
- 没有跨越 server/client 的危险依赖，也没有循环依赖。

阶段 15｜Docker 与本地可复现环境

目标：能将前端静态构建和 Fastify API 打包为可运行镜像，并可靠地管理环境配置。

核心概念清单（逐项验收）

- ☐ 镜像、容器、layer、registry、端口映射、volume、network、entrypoint/cmd 的区别。
- ☐ Dockerfile：锁文件优先、依赖缓存层、.dockerignore、多阶段构建与生产镜像缩小。
- ☐ 非 root 用户、最小运行时依赖、进程信号传递、SIGTERM 优雅关闭。
- ☐ Docker Compose 的服务依赖、健康检查、网络、volume、环境变量。
- ☐ 配置与秘密：运行时注入，不把密钥写入镜像层、构建参数或仓库。
- ☐ Fastify 监听 0.0.0.0、容器端口与宿主端口区别、readiness 与 liveness。
- ☐ Supabase 本地栈依赖 Docker runtime；服务启动顺序与数据库 ready 不等于容器 started。
- ☐ 镜像扫描、依赖更新与可重建性；不要将开发热更新配置原样复制到生产镜像。

必须完成的动手练习

1. 为 apps/api 写 production multi-stage Dockerfile，使用 pnpm lockfile 安装固定依赖。
2. 为 web 生成静态 build 并使用静态文件服务器/适当平台发布；不把 Vite dev server 当生产服务器。
3. 写 Compose 启动 API 与所需依赖，带 healthcheck，并验证 API 能在容器内连接正确数据库。
4. 用 docker inspect 或镜像层检查确认无数据库密码、Better Auth secret 或真实 .env 被烘焙进镜像。

通过标准

- 别人可以从仓库按说明构建和运行镜像。
- 容器停止时进程能优雅结束，健康检查反映真正的服务可用性。
- 生产镜像只包含运行需要的文件、以非 root 运行，并通过环境注入配置。

阶段 16｜GitHub Actions CI/CD

目标：搭建从提交验证到部署的自动化门禁，做到可审计、最小权限、失败可定位。

核心概念清单（逐项验收）

- ☐ workflow、event、job、step、runner、needs、matrix 与 concurrency。
- ☐ 锁定 pnpm/Node 版本，使用 frozen lockfile；用 cache 加速但不依赖缓存保障正确性。
- ☐ CI 质量门禁：format/lint、typecheck、unit/integration test、build、E2E smoke test。
- ☐ GitHub Actions permissions 最小化；secrets/environment protection；fork PR 的不可信代码边界。

- ☐ Cache 与 artifacts 区别；保存测试报告、coverage、Playwright trace 等诊断产物。
- ☐ 迁移与部署顺序：先测试迁移，再应用兼容性策略；需要时采用 expand-migrate-contract。
- ☐ 分支保护、required checks、环境审批、部署回滚与发布标识。
- ☐ OIDC/短期凭据优先于长期静态云密钥（如托管平台支持）。

必须完成的动手练习

1. PR workflow 自动跑 lint、typecheck、Vitest、build。
2. main workflow 执行集成/E2E 测试，并上传失败报告和 trace。
3. 单独设计部署 job：只允许受信任分支/受保护环境部署；限制 GITHUB_TOKEN 权限。
4. 故意使一个检查失败，验证合并门禁和日志足以找到失败步骤。

通过标准

- 全新 runner 能按锁文件安装依赖，不依赖个人机器缓存。
- 密钥从 Secrets/Environment 或 OIDC 获得，不写入代码、日志和 artifact。
- 部署有版本标识、健康验证和可执行的回滚/恢复说明。

边界 / 易错点

CI 的 cache 可能被不同分支或不可信工作流读取；绝不能把秘密存进缓存。部署权限和 pull_request 的权限应分开设计。

阶段 17｜Sentry + OpenTelemetry + 结构化日志

目标：把“出错了”提升到可回答：哪里出错、影响谁、经过哪些服务、如何复现。

核心概念清单（逐项验收）

- ☐ Sentry：错误事件、issue grouping、release/environment、source maps、性能事务与采样。
- ☐ OpenTelemetry：API/SDK、traces/spans、attributes/resources、context propagation、exporter 与 collector。
- ☐ 日志、指标、追踪的区别：事件细节、聚合趋势、一次请求跨服务的因果链。
- ☐ Fastify/Pino 结构化日志：requestId、route、duration、statusCode、error code 等可查询字段。
- ☐ 前后端关联：浏览器 trace / request headers / 后端 span / requestId；按支持情况配置上下文传播。
- ☐ 采样与成本：高吞吐服务不能无条件记录所有 trace；错误和性能采样应有策略。
- ☐ 隐私与安全：屏蔽 token、Cookie、密码、个人敏感信息；审查事件 breadcrumbs、请求体和 source maps。
- ☐ 告警阈值、SLO/错误率/延迟基本思路、部署后回归与故障复盘。
- ☐ 避免重复 instrumentation，明确 Sentry 与 OTel 各自负责的事件采集、传输、展示范围。

必须完成的动手练习

1. 触发一个前端异常并在 Sentry 中按 release/environment 找到它。
2. 在 Fastify 中记录结构化请求日志，并将 requestId 写入错误响应与日志字段。
3. 用 OTel instrument 一条 API 请求，至少看到 server span；按环境导出到可检查的后端/collector。
4. 设计一个“创建任务失败”的排障演练：从 UI 错误定位到 API 请求、trace、DB 操作和权限判断。

通过标准

- 错误、日志与 trace 不只是各自存在；至少一个关键业务流程可以串联定位。
- Sentry 环境/release 配置正确，source maps 上传策略清楚，事件无凭据泄露。
- 能解释 trace/span、attributes、sampling、requestId 和业务错误码的不同用途。
- 有一份短故障复盘：症状、影响、根因、修复、预防措施。

边界 / 易错点

OpenTelemetry JavaScript 各信号的成熟度与 SDK/自动插桩兼容性会随版本变化；实现时查看官方语言状态、具体 instrumentation 与导出器版本，不要假定日志、指标、追踪所有 API 的成熟程度完全一致。

第六部分｜毕业项目：用一个系统验证全部知识

项目建议：多租户 Workboard 任务协作系统

建议用同一个可交付项目贯穿全部阶段，而不是分别做几十个互不关联的小 demo。该项目规模足以逼出权限、缓存一致性、迁移、测试和可观测性问题，又不要求先引入微服务。

必做业务范围

- 身份：注册、登录、登出、会话恢复、未授权访问处理。
- 组织与权限：workspace、成员、角色（owner/admin/member 或相近的最小模型）；支持跨 workspace 隔离测试。
- 项目与任务：CRUD、状态、优先级、负责人、截止时间、评论；有筛选、排序、分页和 URL 可恢复状态。
- UI：响应式布局、loading/empty/error 状态、Dialog/Select/Drawer、可访问的任务表单。
- 数据：关系约束、唯一性、索引、迁移、seed、事务和可重建本地数据库。
- API：TypeBox schemas、OpenAPI 文档、稳定错误 envelope、请求 ID、分页契约。
- 测试：业务规则单元测试、Fastify API 集成测试、关键用户旅程 Playwright E2E。
- 部署工程：pnpm monorepo、Turborepo pipeline、Docker 构建、GitHub Actions PR 门禁。
- 监控：Sentry 错误事件、结构化后端日志、至少一条跨层 trace 与诊断说明。

推荐的数据模型（按业务需要调整）

表	核心字段 / 约束重点
workspace	id、name、created_at；组织/租户边界。
workspace_member	workspace_id、user_id、role；复合唯一约束；成员关系与角色。
project	id、workspace_id、name；索引 workspace_id；外键删除策略。
task	id、workspace_id、project_id、title、description、status、priority、assignee_user_id、created_by、due_at、created_at、updated_at。

表	核心字段 / 约束重点
comment	id、task_id、author_user_id、body、created_at；按 task_id 与时间排序查询。
Better Auth tables	由 Better Auth 当前推荐 schema / adapter 规则管理；明确迁移的唯一所有者。

前后端边界建议

层	职责	不应承担的职责
React Router	URL/页面层级、路由切换、loader/action 的导航集成。	不要维护第二套与 TanStack Query 不同步的数据缓存。
TanStack Query	服务端缓存、请求状态、mutation、失效与乐观更新。	不要当成所有表单输入/弹窗状态容器。
Zustand	少量跨组件 UI 状态，如侧栏、全局面板、临时 UI 选择。	不要复制服务端列表、会话或可从 URL 恢复的状态。
RHF + Zod	表单生命周期、用户体验校验、输入/输出转换。	不要作为授权或可信数据校验。
Fastify + TypeBox	API 边界校验、身份提取、资源授权、业务流程、错误契约。	不要仅信任前端类型或前端已通过校验。
Drizzle	有类型的数据库访问、事务、查询组合。	不要替代 schema 设计、数据库约束或 EXPLAIN。
Postgres / RLS	持久化、一致性、约束；在采用直接访问时配置 RLS 防护。	不要假定 RLS 自动覆盖所有服务端管理路径。

建议的交付里程碑

1. M1 可运行骨架：repo 结构、基础页面、API health、环境配置、README。
2. M2 数据闭环：schema、迁移、seed、Drizzle 查询、列表/详情 API。
3. M3 身份与租户隔离：Better Auth、成员模型、每条资源 API 授权和越权测试。
4. M4 可用 UI：路由、Query 缓存、Zustand UI 状态、RHF + Zod、shadcn/Base UI。
5. M5 质量门禁：Vitest、Playwright、CI 报告、构建和类型检查。
6. M6 运行与排障：Docker、部署、Sentry、OTel、结构化日志、一次故障复盘。

项目完成定义（Definition of Done）

- ☐ 全新 clone 按 README 可以配置环境、启动服务、重建 schema、加载 seed、运行测试。
- ☐ 任何 workspace 跨租户访问都由自动化测试验证为拒绝；安全关键路径无待办 TODO。
- ☐ API OpenAPI 文档与真实路由一致；前端 client 类型生成在 CI 有检查。
- ☐ 关键路径有 unit/integration/E2E 多层测试；失败可查看报告与 trace。
- ☐ 容器不内嵌秘密；PR 门禁、部署权限和数据库迁移责任清晰。

第七部分｜掌握度量表：按证据推进，而不是按感觉推进

为每个技术建立一页自评记录。建议把每个核心清单项标为：0 = 不了解；1 = 能解释但不能独立实现；2 = 能独立实现；3 = 能覆盖错误路径、调试并说明取舍。只有 2 或 3 才算“掌握”。

评分	定义	下一步
0	无法正确解释或辨认。	回到官方文档与最小示例，先建立概念图。
1	能解释，但实现需要逐步跟教程。	关闭教程重做；把核心 API 写成小实验。
2	独立实现 happy path，并有基本测试。	主动引入边界情况/故障，验证错误路径。
3	可定位真实故障、解释取舍、设计可维护实现。	记录为可复用模式，必要时写 ADR / 内部指南。

通过门槛

- ☐ 单个技术：核心清单至少 90% 的条目达到 2 分或 3 分。
- ☐ 安全关键项：全部达到 3 分或经同行审查通过，不以总体覆盖率抵销。
- ☐ 技术整合：至少完成一次从前端到数据库的端到端功能，且至少一种失败路径有自动化测试。
- ☐ 工程化：别人能从文档复现开发、测试、构建和迁移；失败时有可读日志。
- ☐ 长期维护：遇到新版本升级时能阅读 migration guide / changelog，并设计兼容验证，而不是默认升级无风险。

每个技术模块的复盘模板

```
技术 / 模块：
我能不看文档解释的 5 个核心概念：
1.
2.
3.
4.
5.

我独立实现的功能：
我写过的失败路径测试：
我解决过的一个真实 bug：
我考虑过但没有选择的替代方案，以及原因：
官方文档 / 版本变更链接：
尚未掌握的核心清单与下一步：
```

值得长期保留的工程文档

- README：安装、环境变量、启动、迁移、seed、测试、构建与故障排查。
- ADR（架构决策记录）：为什么选 Data Mode、Query/Zustand 分工、认证源、迁移源、OpenAPI 生成方案。
- API 规范：错误格式、分页、日期/时区、身份/权限、版本演进。
- 安全清单：secret 来源、RLS 策略、租户隔离、CORS/Cookie、日志脱敏。
- 运行手册：health endpoints、告警、Sentry 查询、trace 查找、数据库恢复与回滚步骤。

第八部分 | 高频误区与架构原则

误区	正确原则 / 检查方式
将 React state、URL、Query cache、Zustand、RHF 的状态混在一起。	按状态拥有者拆分：URL 可分享状态给 Router；服务端状态给 Query；表单状态给 RHF；少量全局 UI 状态给 Zustand。
前端 Zod 通过，所以 API 可以信任请求。	所有外部输入在 Fastify 边界用 TypeBox/JSON Schema 重新验证，业务层还要检查授权/状态转移。
已经登录，所以能访问传进来的 taskId。	每次读写都要验证资源归属与 workspace membership；随机 UUID 不是授权机制。
有 RLS，就不需要 API 授权；或 service role 会自动受 RLS 保护。	厘清数据库访问身份。API 层实施授权，RLS 只在对应数据库角色与策略执行路径内生效。
Drizzle ORM 会自动保证数据模型正确。	数据库约束、事务、索引、迁移及查询计划仍需设计和验证。
同一 schema 同时由 Drizzle 与 Supabase CLI 任意改。	选定唯一 schema change source of truth，并在 CI 检查迁移可重放。
Swagger UI / Scalar 产生文档后，API 契约就稳定了。	需要请求/响应 schema、错误模型、版本策略和兼容性检查；文档必须跟真实路由一致。
覆盖率高就代表质量高。	覆盖率结合业务风险、断言质量、错误路径、隔离和集成测试看。
E2E 挂了就加 waitForTimeout。	查看 trace、网络、后端日志与共享数据，使用 auto-wait 和可靠状态断言。
Docker 镜像能跑就算生产化。	还要检查非 root、信号处理、健康检查、资源限制、secret 注入与可重建性。
Sentry、日志和 OTel 都开了就能排障。	需要 release/requestId/trace context 能关联，并有实际演练证明排障路径有效。

附录 A | 官方资料索引

按本路线的模块组织。先阅读 Getting Started / Concepts，再用 API Reference 查边界情况。具体 API、默认值和版本兼容性以实施时打开的最新官方文档为准。以下链接均为官方项目文档或官方维护仓库。

基础语言与工具

- React Learn — 组件、状态、Effect 与组合
- React + TypeScript — React 组件与 Hooks 的类型实践
- TypeScript Handbook — 类型系统与控制流分析
- Vite Guide — 开发服务器、构建、配置与环境变量
- Node.js Learn — Node 运行时、异步、模块与诊断

前端状态与 UI

- React Router Modes — Data / Declarative / Framework 模式
- TanStack Query React Docs — Query、Mutation、缓存与 Devtools
- TanStack Query Important Defaults — staleTime、gcTime 与自动刷新
- Zustand Docs — Store、selector、中间件
- shadcn/ui Docs — 组件源码、主题、安装与变体
- Base UI Docs — Base UI React primitives
- React Hook Form — 表单状态和字段注册
- Zod Basics — 运行时 schema、parse 与类型推断
- Tailwind CSS Docs — Utilities、主题与响应式样式

API 与数据库

- Fastify Latest Docs — 插件、生命周期、验证、序列化
- Fastify Validation & Serialization — JSON Schema 与响应序列化
- TypeBox Repository — TypeBox runtime schema 与 Static 类型
- OpenAPI Specification — API 描述规范
- @fastify/swagger — 从路由 schema 生成 OpenAPI
- Scalar Fastify API Reference — Scalar API reference 集成
- Supabase Docs — 数据库、RLS、CLI、Storage 与环境
- Supabase Local Dev Workflow — 本地可复现开发流程
- Supabase Database Migrations — schema 变更版本化
- Drizzle ORM Overview — 类型化 SQL/ORM 使用方式
- Drizzle Migrations — 迁移策略与 drizzle-kit
- Drizzle Relations — 关系查询与 SQL join
- Better Auth Introduction — 认证能力与架构概述
- Better Auth Fastify Integration — Fastify handler 与跨域设置
- Better Auth Session Management — Session 与 Cookie 生命周期

测试与工程化

- Vitest Guide — 测试配置、mock、测试运行
- Vitest Coverage — 覆盖率收集与解读
- Playwright Introduction — E2E 测试入门
- Playwright Fixtures — 测试隔离与 fixture
- pnpm Workspaces — workspace 协议与 monorepo
- Turborepo Docs — 任务图、缓存与 monorepo
- Docker Get Started — 镜像、容器和 Compose
- Docker Build Best Practices — 构建缓存与镜像实践
- GitHub Actions Docs — Workflow、job、权限与 artifacts
- GitHub Actions Cache Concepts — 缓存策略与安全边界

观测与排障

- Sentry JavaScript / React — 前端错误与性能监控
- Sentry Node.js — Node 服务端错误与性能监控
- OpenTelemetry JavaScript — JS SDK、instrumentation、exporter 与状态
- OpenTelemetry Getting Started — Node/browser 入门示例

附录 B | 建议阅读文档的方式

- 先读概念页，画出组件关系图；随后读最常用 API，不要一开始就通读所有 API Reference。
- 每次阅读都变成一个可执行的小实验，并写下一个“错误实验”：故意设置错误配置/输入，观察失败位置与错误信息。
- 升级依赖前看官方 migration guide、release notes 与 peer dependency；锁定版本并通过项目测试后再合并。
- 遇到第三方博客与官方文档冲突时，先核对所用主版本；旧教程可能对应不同 API、配置语法或默认值。
- 将实际项目中的决策写入 ADR/README：原因、替代方案、风险、回滚方式与验证证据。